

Sujet d'examen UE NFE204

Année universitaire 2023-2024

Examen première session : 22 juin 2024

Consignes

Durée de l'examen : 3h00; Calculatrice autorisée ; Documents autorisés : 5 pages recto-verso de note personnelles.

Les exercices sont indépendants les uns des autres. Merci de soigner votre écriture.

Ce sujet contient 4 pages.

Important : nous n'évaluons pas les réponses par la syntaxe. Ne vous inquiétez pas d'utiliser un point-virgule à la place d'un point d'exclamation ou autres détails mineurs

Première partie : documents structurés (6 pts)

Nous approchons des jeux olympiques de Paris 2024, événement majeur, mais source probable de désordres importants. Les organisateurs décident de mettre en place un système de co-voiturage à destination des sites majeurs, afin d'éviter les encombrements.

On pense au départ d'une base relationnelle sera suffisante, et on adopte le modèle suivant (très simplifié bien sûr). Les clés primaires sont en **gras**, les clés étrangères en *italiques*.

- Personne (**id**, nom, domicile)
- Site (**id**, nom)
- Trajet (**id**, *idConducteur*, *idSite*, date)
- Passager (*idTrajet*, *idPassager*)

Voici également un tout petit échantillon de la base.

id	nom	domicile
1	Albert	Melun
2	Aline	Versailles
3	Ktie	Créteil

La table *Personne*

id	nom
100	Stade de France
101	Parc de Versailles
102	Grand Palais

La table *Site*

id	<i>idConducteur</i>	<i>idSite</i>	date
A	1	100	30/07
B	3	100	4/08
C	1	101	31/07

La table *Trajet*

id	<i>idPassager</i>
A	2
A	3
B	1
C	2

La table *Passager*

On s'aperçoit rapidement que cette représentation impose beaucoup de jointures et ne passe pas à l'échelle. On décide de remplacer la base relationnelle par une base Cassandra.

1. Voici une première table Cassandra pour représenter les trajets.

```
create table Trajet1 (idTrajet text,
                     date date,
                     site frozen<Site>,
                     conducteur frozen<Personne>,
                     passagers set< frozen<Personne>>,
                     primary key idTrajet );
```

Donnez l'exemple d'un document JSON correspondant à la structure, de la table Trajet1, avec les informations du trajet A de la base relationnelle ci-dessus.

- Voici la proposition d'une autre modélisation, avec une table Trajet2 et une table Passager dont la clé est *composite*

```
create table Trajet2 (idTrajet text,
                    date date,
                    site frozen<Site>,
                    conducteur frozen<Personne>
                    primary key (idTrajet);

create table Passager (idTrajet text,
                    idPassager text,
                    passager frozen<Personne>,
                    primary key (idTrajet, idPassager );
```

Rappelons ce qu'est une clé composite en Cassandra, d'après la documentation : *A compound primary key consists of the partition key and the clustering key. The partition key determines which node stores the data. Rows for a partition key are stored in order based on the clustering key.* Quelles affirmations sont vraies ?

- Les lignes de Trajet2 et les lignes de Passager sont sur le même serveur
 - Les passagers d'un même trajet sont tous sur un seul serveur
 - Les passagers stockés sur un serveur sont triés sur leur identifiant
 - Les passagers d'un même trajet sont stockés contiguement
- Est-il possible de convertir la table Trajet1 vers la table Passager avec un traitement Map Reduce ? Expliquez comment, en indiquant ce que font les fonctions de Map et de Reduce.

Deuxième partie : recherche d'information (6 pts)

L'application enregistre des commentaires déposés par les clients sur les conducteurs, et réciproquement. Voici un échantillon.

- c_1 Conduite dangereuse, et conducteur bavard. À éviter.
- c_2 Pour la conduite ça ne va pas : le conducteur (sympa par ailleurs) réussit à être à la fois lent et dangereux
- c_3 Trop dangereux ! Je veux bien payer pour un co-voiturage, mais pas avec un conducteur dangereux.
- c_4 Sympa, mais le conducteur est très bavard et lent.
- c_5 Il est vraiment bavard de chez bavard. Heureusement, sympa et rien de dangereux dans sa conduite.

On va se limiter au vocabulaire (dangereux, bavard, lent, sympa). (NB : on suppose une phase préalable de normalisation qui élimine les pluriels, majuscules, trouve que "sympa" et "sympathique", "dangereux" et "dangereuse", sont les mêmes mots, etc.)

- Donnez une matrice d'incidence contenant les tf, et un tableau donnant les idf (sans le log), pour les termes précédents. Placez les termes en ligne, les commentaires en colonne.

Solution :

	c1	c2	c3	c4	c5
dangereux 5/4	1	1	2	0	1
bavard 5/3	1	0	0	1	2
lent 5/2	0	1	0	1	0
sympa 5/3	0	1	0	1	1

- Donnez les normes des vecteurs représentant ces commentaires.

Solution : Les normes

$$\|c_1\| = \sqrt{1+1}; \|c_2\| = \sqrt{3}; \|c_3\| = \sqrt{4}; \|c_4\| = \sqrt{3}, \|c_5\| = \sqrt{6}$$

- Donner les résultats classés par similarité cosinus basée sur les tf (on ignore l'idf) pour les requêtes suivantes. Expliquez brièvement la raison du classement pour le premier document.

- bavard;
- lent et sympa;
- dangereux et bavard.

Solution : Les cosinus (requête non normalisée).

- bavard : $c1 : \frac{1}{\sqrt{2}}$; $c4 : \frac{1}{\sqrt{3}}$; $c5 : \frac{2}{\sqrt{6}}$;
- lent et sympa :
 $c2 : \frac{1+1}{\sqrt{3}}$; $c4 : \frac{1+1}{\sqrt{3}}$ $c5 : \frac{1}{\sqrt{6}}$;
- dangereux et bavard
 $c1 : \frac{1+1}{\sqrt{2}}$; $c2 : \frac{1}{\sqrt{3}}$; $c3 : \frac{2}{\sqrt{3}}$ $c4 : \frac{1}{\sqrt{3}}$ $c5 : \frac{1+2}{\sqrt{6}}$;

4. Pour la requête “lent et sympa”, et les documents c2 et c4, qu’est-ce qui change si on ne met pas de restriction sur le vocabulaire (tous les mots sont indexés) ?

Solution : Dans ce cas le plus petit document (c4) l’emporte car sa norme est plus petite. Intuitivement : il parle plus *spécifiquement* de lent et de sympa que c2.

Troisième partie : MapReduce (3 pts)

Nous recevons un flux de commentaires dans un fichier `trajets.csv`. Il contient l’identifiant du conducteur, du client, et le commentaire. Voici le début.

```
1, 101, "Voiture bruyante"
1, 103, "Une vraie épave"
2, 101, "La voiture est propre, plus que le conducteur..."
..
```

Utilisant Pig latin, on charge cette collection avec la commande suivante :

```
trajets = LOAD 'trajets.csv' as (idConducteur, idClient, commentaire);
```

Et on exécute le script Pig suivant :

```
coll1 = filter trajets by contains(commentaire, "voiture")
coll2 = group coll1 by idConducteur;
coll3 = foreach coll2 generate group, COUNT(coll1.commentaire);
```

1. Expliquez le résultat produit par ce script.
2. Comment ce script peut-il être traduit en MapReduce ? Donnez la fonction de Map et la fonction de Reduce, sous la forme de votre choix.
3. Quelle serait la requête SQL correspondant à ce script Pig ?

Quatrième partie : systèmes distribués (3 pts)

1. Sachant que Cassandra organise la distribution des données selon la technique du hachage cohérent, expliquez l’algorithme qui place un document de la table `Passager` sur un serveur.
2. Parmi les requêtes ci-dessous, lesquelles peuvent être routées vers un seul serveur ?
 - (a) Les trajets vers Versailles
 - (b) Les passagers du trajet $t12$ (c’est son identifiant)
 - (c) Les sites visités par le passager $p988$ (c’est son identifiant)
3. À l’instant t , les valeurs de hachage de deux trajets T_1 et T_2 sont identiques. Peuvent-ils être placés sur deux serveurs différents à un instant $t + \Delta$, sachant qu’entretemps plusieurs nœuds ont été ajoutés au système Cassandra ?

Cinquième partie : question de cours (2 pts)

1. Comment expliquer l'expression "index inversé" dans la terminologie des moteurs de recherche ?
2. Rappeler le principe de *data locality*